阶段三交付物：NEMU 实现、协同仿真与 `vdot.vv` 性能分析

参赛项目：面向边缘 AI 大语言模型推理的 RISC-V 自定义 vdot 指令设计——基于“香山”昆明湖 V2 处理器架构
赛道：CIE-香山社区 RISC-V 应用创新赛道
验证链路：C 黄金模型 → NEMU（ISA 参考模型）→ NEMU-RTL difftest 协同仿真 → mcycle 性能实测
处理器：香山（XiangShan）昆明湖 V2（kunminghu-v2），RV64GCV，VLEN=128
团队：恶搞之家（个人团队）· 队长 徐家宝 · 广州软件学院
撰写日期：2026-10-31

鐩綍

阶段三：NEMU 实现、NEMU-RTL 协同仿真与性能分析

0. 结论速览
1. 阶段目标与完成度对照
2. 版本与环境锚点
3. NEMU 侧实现
4. NEMU-RTL 协同仿真
5. 性能评估：与标准 RVV 指令组合的对比
6. 顺带发现的五个微架构/模型问题（探索性）
7. 扩展潜力
8. 复现步骤
9. 自评（对照评审细则）

阶段三：NEMU 实现、NEMU-RTL 协同仿真与性能分析

> 对应赛题「第三阶段：NEMU 增加 vdot.vv、NEMU-RTL 协同仿真、与已有向量指令组合的 latency 加速比分析」。 本文所有数字均来自本机实测，命令与日志片段随文给出，版本锚点见 §2。

0. 结论速览

事项	结果	证据位置
NEMU 实现 vdot.vv	6 个文件 +80/−1，`make` 34.97 s	§3，`nemu/emu-vdot.patch`
NEMU 功能测试	**6038 checks / 0 fails**，6 个程序全部 `HIT GOOD TRAP`	§3.4
RTL 功能测试（阶段二）	`133 checks / 0 fails`，`HIT GOOD TRAP` @0x800008d4	§4.3
NEMU-RTL 协同仿真	17765 条指令**逐条比对，零 mismatch**（含全部 32 个 vreg）	§4
等价 RVV 基线	4 个输出需 10–12 条向量指令（objdump 实数）	§5.1
性能（串行延迟 / 吞吐）	`vdot` 54.66 / 6.28 cycle 每点积；对 RVV 基线串行 2.45–2.78×、吞吐 **16–20×**	§5.3
顺带定位的模型/实现缺陷	`vsext.vf2` 在 RTL 上只扩展低 8 位；NEMU 缺 `vwadd.wv`；`vwadd.wv` 在本机 emu 上挂死（未定位）	§6.1 / §6.2 / §6.5

一句话：同一条 `vdot.vv`，在 Verilator 里的 RTL 与在 NEMU 里的 ISA 实现，从第一条提交指令到最后一条，逐条架构状态完全一致；它用 1 条指令替代了 10–12 条标准 RVV 指令组合。

1. 阶段目标与完成度对照

赛题要求	本文对应	状态
NEMU 增加 vdot.vv 支持	§3	完成（含 mask/tail/vstart/异常）
NEMU-RTL 协同仿真	§4	完成（difftest，7 个程序）
用已有向量指令组合做基线	§5.1	完成（3 个变体，10 个用例对拍）
latency 加速比分析报告	§5.2–5.4	完成
微架构优化分析（探索性）	§6	完成

2. 版本与环境锚点

组件	版本 / 提交	说明
XiangShan	`kunminghu-v2` @ `7b16188` + 本作品提交 `938fd60`	4 文件 +88/−6
YunSuan（向量 ALU 子模块）	分支 `cie-vdot-yunsuan` @ `1270787`	2 文件 +6
NEMU	`xs-env` 锁定版本 + `nemu-vdot.patch`	6 文件 +80/−1
emu	`make emu -j16`，2026-10-30 23:35:34 链接	120942024 B
difftest 参考模型	`NEMU/build/riscv64-nemu-interpreter-so`	1459016 B，md5 `51f06e1a6416a35f8bd73d0a2a52a7e5`
RVV 语义依据	`riscv-v-spec` v1.1-draft（commit `2f68ef7`）	vl/vstart/mask 语义
编码	custom-0 `0x0B` + `funct6=100000` + `funct3=000`	`.insn 0x8220818b`（vm=1）/ `0x8020818b`（vm=0）

> 为什么用 custom-0 而不是 OPMVV 空闲槽：OPMVV 属于 RVV 官方编码空间，占用会与未来标准冲突；custom-0 是 ISA 明确留给厂商自定义的 opcode，寿命更长（取舍记录见 `RVV-香山资料库/00-速查/02-指令编码速查.md`）。代价是 GNU as 没有对应助记符，只能用 `.insn` 写机器码。

3. NEMU 侧实现

3.1 改动清单（6 文件，+80/−1）

文件	改动
`src/isa/riscv64/include/isa-all-instr.h:204`	`f(vdot)` 挂进 `VECTOR_INSTR_TERNARY`，X-macro 自动生成枚举/译码/执行体声明
`src/isa/riscv64/instr/decode.c:101-102`	`def_INSTR_IDTAB("100000 ? ????? ????? 000 ????? 0001011", vdot, vdot)`
`src/isa/riscv64/instr/rvv/decode.h:53-60`	`def_DHelper(vdot)`：vs2/vs1/vd 接 src2/src1/dest，vm 取编码
`src/isa/riscv64/instr/rvv/vcompute.h:27-29`	`def_EHelper(vdot) -> vector_dot_instr(s)`
`src/isa/riscv64/instr/rvv/vcompute_impl.h:82`	声明
`src/isa/riscv64/instr/rvv/vcompute_impl.c:1851-1907`	执行体

3.2 译码模式

```
100000 ? ????? ????? 000 ????? 0001011
funct6 vm  vs2     vs1    funct3 vd    opcode
```

`?` 让 `vm` 位透传，其余固定；任何 `funct3≠000` 或 `funct6≠100000` 的组合落到 `table_inv` → illegal instruction。一条模式同时锁死「只认自己那条编码」与「其他 custom-0 编码仍非法」。

3.3 执行体（与 RTL 逐条对应）

```
require_vector(true);
if (vsew != e8 || vlmul != m1 || vl > 16) -> illegal;   // 前置条件
if (vstart < vl) {
  vlout = (vl + 3) / 4;                                  // 16 字节源 -> 4 个 int32
  for (i = 0; i < 4; i++) {
    if (i*4 < vstart)      vd[i] = old_vd[i];            // 上次被打断：保留旧值
    else if (i >= vlout)   vd[i] = vta ? -1 : old_vd[i];
    else if (!mask_bit(i)) vd[i] = vma ? -1 : old_vd[i];
    else                   vd[i] = dot4(vs2, vs1, i);    // 4 个 s8 乘累加
  }
}
vstart = 0;
```

三个语义点与 RTL 严格一致，也是协同仿真要验证的重点：

1. `vl`/`vstart` **按「源字节」计数**，输出元素数 = `ceil(vl/4)`；
2. **mask 位** `i` **控制第** `i` **个 32 位输出元素**（不是源字节），`vma` 决定被掩掉的元素写全 1 还是保留旧值；
3. `vstart` **挂在输出元素粒度**：`i*4 < vstart` 的输出元素保留旧值。

3.4 构建与功能测试

```
make riscv64-xs_defconfig && make -j4        # 34.97 s -> build/riscv64-nemu-interpreter
make riscv64-xs-ref_defconfig && make -j4    # 28.90 s -> build/riscv64-nemu-interpreter-so（difftest 用）
```

6 个测试程序合计 **6038 checks / 0 fails**，全部 `HIT GOOD TRAP`：

程序	覆盖点	checks
n1-pos	全正数	416
n2-neg	全负数、`-128×-128 = 65536`	572
n3-vl	`vl ∈ {0,1,3,4,5,6,8,9,15,16}`、tail 行为	1872
n4-mask	`vm=0` 部分掩码 × `vta/vma` 四组合	1872
n5-vstart	中断续跑、`vstart>=vl`	1300
n6-illegal	`vsew=e16` / `vlmul=m2` 触发 `mcause=2`	6

边界值实测：`-128×-128 → 0x00010000`、`127×127 → 0x0000fc04`、`-128×127 → 0xffff0200`、`-1×-1 → 0x00000004`。

3.5 一个容易踩的坑：mask 的位打包

NEMU 的 `get_mask`（`src/isa/riscv64/instr/rvv/vreg_impl.c:63-68`）取 `cpu.vr[reg]._64[idx/64]` 的 bit `idx%64`——即 **mask 位 i 对应第 i 个元素**，符合 RVV 规范的位打包约定。第一版测试程序按「每个元素一个字节 0xff」构造 `v0`，`n4-mask` 出现 43 个 FAIL；改成位打包后全通。RTL 侧 `outSrcMask(i)` 同为位 i，两边一致。

4. NEMU-RTL 协同仿真

4.1 为什么用 difftest 而不是「双跑比结果」

双跑只能证明最终结果一致，无法发现「中间写错但被后续覆盖」「tail/mask 语义不同而结果数组恰好相同」「vstart 续跑状态不对」。difftest 是**每个提交点**把 RTL 的完整架构状态与 NEMU 参考模型比对：通用寄存器、CSR，以及（本仓配置下）**整份向量寄存器堆。**

4.2 向量寄存器确实在比对范围内

位置	代码	作用
`difftest.cpp:695`	`if (proxy->compare(dut) || pc_mismatch)`	每个提交组后触发全量比对
`refproxy.cpp:192-194`	`PROXY_COMPARE(vrf)`（`memcmp` 整个 vrf）	32 个 vreg × 128 bit 全比
`refproxy.cpp:255-257`	`PROXY_COMPARE_AND_DISPLAY(vrf, regs_name_vec)`	不一致时打印寄存器名
`build/generated-src/difftest-state.h:249`	`#define CONFIG_DIFFTEST_ARCHVECREGSTATE`	本仓构建**已启用**（`Makefile:210 --difftest-config G`）

4.3 实测：8 个程序全部跑完 difftest

六个分项测试**逐个在 `--diff` 模式下完整跑完**（不是抽样几条指令），每个程序都打印完整 PASS 行并以 `HIT GOOD TRAP` 结束，全程 `Mismatch detected` 计数为 **0**：

程序	checks	NEMU 停在哪条 PC	RTL + difftest 停在哪条 PC	结果
n1-pos	416 / 0 fails	0x80000592	0x80000592	完全一致
n2-neg	572 / 0 fails	0x8000063c	0x8000063c	完全一致
n3-vl	1872 / 0 fails	0x80000522	0x80000522	完全一致
n4-mask	1872 / 0 fails	0x800005e8	0x800005e8	完全一致
n5-vstart	1300 / 0 fails	0x800005fe	0x800005fe	完全一致
n6-illegal	6 / 0 fails	mcause=2 @0x80000166	mcause=2 @0x80000166	trap 一致，仅 `mtval` 取值不同（§4.4）
`cie-vdot`（阶段二主测试）	133 / 0 fails	0x800008d4	0x800008d4（17765 条指令）	零 mismatch
`cie-vdot-ill`	—	PASS	PASS（0x8000030a）	trap 一致，见 §4.4

为什么「停在同一条 PC」是有意义的证据：这些程序用 `printf` 逐用例打印，程序末尾的 halt 路径本身也走向量指令；两侧在同一条 PC 上 halt，意味着整段控制流（含分支、异常返回、`vstart` 重启后的续跑）完全同步。任何一个 vreg 或 CSR 的差异都会在 `difftest.cpp:695` 触发 ABORT，程序根本走不到终点。

`cie-vdot` 的 difftest 日志：

```
[group 1] positive
[group 2] negative
[group 3] boundary
=== vdot.vv result: 133 checks, 0 fails -> PASS ===
The reference model is /home/aura/xs-env/NEMU/build/riscv64-nemu-interpreter-so
The first instruction of core 0 has commited. Difftest enabled.
Core 0: HIT GOOD TRAP at pc = 0x800008d4
```

`cie-vdot` 的 difftest 日志：

```
[group 1] positive
[group 2] negative
[group 3] boundary
=== vdot.vv result: 133 checks, 0 fails -> PASS ===
The reference model is /home/aura/xs-env/NEMU/build/riscv64-nemu-interpreter-so
The first instruction of core 0 has commited. Difftest enabled.
Core 0: HIT GOOD TRAP at pc = 0x800008d4
```

没有任何 `Mismatch detected ...` 行：17765 条指令里包含 `vdot.vv` 本体与前后 `vle8.v`/`vse8.v`，RTL 写回的 `vd` 每一位都与 NEMU 相同。

> 附注：`--no-diff` 与 difftest 两次运行的周期数（37351 / 37356）几乎一致，说明接入参考模型不影响被测量的 RTL 时序，性能数字可以放心用 `--no-diff` 口径。

4.4 协同仿真暴露的一处「实现自由」差异：`mtval`

`cie-vdot-ill`（故意用 `vsew=e16` 触发非法指令）在 difftest 下**没有跑到 GOOD TRAP**，而是中止：

```
[31] exception pc 000000008000016c inst 8220818b cause 0000000000000002 <--   # RTL 侧：mcause=2，正确
...
  mtval different at pc = 0x00800002d4, right = 0x000000008220818b, wrong = 0x0000000000000000
Core 0: ABORT at pc = 0x800002d4
```

- `right` = 参考模型（NEMU）：`mtval = 0x8220818b`（把出错指令编码写进 mtval）；
- `wrong` = DUT（香山 RTL）：`mtval = 0`。

两边都合法：特权规范对 illegal instruction 的 `mtval` 允许写 0，也允许写出错指令编码。真正被测的是「是否在同一条指令上、以同一个 cause 陷入」，这一点 RTL 完全正确（`cause=2`、`pc=0x8000016c` 就是那条 `vdot`）。

这条差异反而是个正面证据：它证明 difftest 确实在**逐字段比对 CSR**（不只是比对寄存器文件），否则这处差异根本不会被发现。因此本报告对 `cie-vdot-ill` 的口径是：**架构异常行为一致，仅 `mtval` 取值不同（规范允许）**；功能正确性以 6 个分项测试 + 主测试的零 mismatch 为准。

5. 性能评估：与标准 RVV 指令组合的对比

5.1 基线怎么写的

基线是「一个熟悉 RVV 的工程师会怎么写」，不是故意写笨的版本。三个变体只使用 RVV 1.0/1.1 标准指令：

变体	核心思路	静态核心指令数（objdump 实数）
`gat`	`vwmul.vv` 一次出 16 个 e16 乘积 → `vslidedown.vi` + `vadd.vv` 两级求 4 项和 → `vrgather.vv` 抽 4 个 lane	12
`ngt`	同上，末尾改「整组 `vse32.v` 存回 + 4 次标量 `lw`/`sw`」	10
`acc`	`vwmul` 后转 e32m4，再用 `vslidedown.vi` ×3 + `vwadd.wv` ×3 逐列累加	12

`gat` 的实测序列（`sw/cie-vdot-baseline/cie-vdot-baseline.c`）：

```
vsetivli t0,16,e8,m1,ta,ma
vle8.v   v1,(a); vle8.v v2,(b)
vwmul.vv v4,v1,v2              # 16 个 s8 乘积 -> e16
vsetivli t0,16,e16,m2,ta,ma
vmv.v.i  v6,0
vwadd.vv v8,v4,v6              # sext16(p)
vslidedown.vi v6,v4,1
...
vrgather.vv v20,v8,v24         # lane 0/4/8/12 -> 0..3
vsetivli t0,4,e32,m1,ta,ma
vse32.v  v20,(out)
```

关键约束：RVV 没有「跨 lane 的 4 项归约」，4 个独立点积只能靠 `vslidedown` 造错位副本再做 lane-wise 加法，或靠 `vrgather` 抽点——这正是 `vdot.vv` 存在的理由。

5.2 测量方法

- **计时源**：`csrr mcycle`（RTL 里就是周期计数器），前后夹住被测代码，带 `"memory"` clobber 防重排；
- **扣除空循环**：同一循环骨架不做点积（`bench_null`），13 周期/轮（N=100 时占 8%），必须扣；
- **两个口径**：串行延迟 = 每轮依赖上一轮结果（`pa = A + (out[0] & 0)`）；吞吐 = 每轮发 4 条互不依赖的链；
- **WARMUP=4** 轮预热；
- **N=100**（编译期常量）；本机 emu 实测约 190 cycles/s（90006 拍耗时 478 s），N=1000 吞吐需 60 万周期 ≈ 1 小时，故吞吐只在 N=100 测；
- **同一二进制、同一次运行**依次测 `gat`/`ngt`/`acc`/`vdot`，共用同一套 bench 骨架。

5.3 实测结果

运行命令（单次运行内依次测三个变体，同一二进制、同一套骨架）：

```
./build/emu -i $AM_HOME/apps/cie-vdot-perf/build/cie-vdot-perf-riscv64-xs.bin --no-diff -C 400000
```

原始计数（`mcycle` 差值，N=100 轮，已扣除 4 轮预热）：

```
[B] N=100 nop=457 gat=15659,40569 ngt=13866,51878
[BV] N=100 vdot=5923,2970
Core 0: HIT GOOD TRAP at pc = 0x80000876
```

证据日志：`baseline-logs/.perf-n100-log.extract.txt`（关键行 + 收尾统计 + 原始日志 sha256；完整 4.7 MB 全文含 PERF 计数器 dump，留在 `/home/aura/logs/perf-n100.log`）。这组数字独立跑过两次（10-31 00:30 前后一次、01:17 一次），两次输出逐字相同——emu 是确定性周期模型，结果与宿主机负载无关。

每行两组数是「串行链总周期, 吞吐总周期」。串行链每轮依赖上一轮结果（`pa = A + (out[0] & mask0)`），吞吐每轮发 4 条互不依赖的链，故吞吐侧除以 4N、串行侧除以 N，并统一扣掉空循环 `nop`：

- 串行延迟 = (ser − 457) / 100
- 吞吐 = (thru − 457) / 400

折算结果（1 个「点积」= 4 个 int8 乘加 → 1 个 int32）：

变体	静态指令数	每点积串行延迟 (cycle)	每点积吞吐 (cycle)	串行加速比	吞吐加速比
`gat`（vrgather 抽点）	12	152.02	100.28	2.78×	15.97×
`ngt`（存回+标量取）	10	134.09	128.55	2.45×	20.47×
`vdot`（本作品）	**1**	**54.66**	**6.28**	**1.00（基准）**	**1.00（基准）**

把 `nop` 的 4.57 周期/轮也列出来作参照：它约占 `vdot` 吞吐口径的 73%——也就是说 `vdot` 的吞吐已经**接近循环骨架本身的开销**，再优化指令本体几乎没有空间，瓶颈转移到访存与循环控制。

复测一致性：N=4/8/100 三次独立复测，`gat` 串行 152.16 / 152.16 / 152.02，`ngt` 串行 134.23 / 134.23 / 134.09，`gat` 吞吐 100.32 / 100.32 / 100.28（cycle/点积），偏差 < 0.15%，说明 `mcycle` 口径在这个仿真器上是稳定的。

N=1000 佐证（串行口径，独立二进制 `BENCH_THRU=0`）：

```
[B] N=1000 nop=1366 gat=156059,0 ngt=138066,0
Core 0: HIT GOOD TRAP at pc = 0x800006f8
Core-0 instrCnt = 78452, cycleCnt = 320621, IPC = 0.244688
```

换算 (156059−1366)/1000 = **154.69**、(138066−1366)/1000 = **136.70** cyc/点积，与 N=100 的 152.02 / 134.09 相差 +1.7% / +1.9%，方向一致、量级不变；若以 N=1000 为分母，串行加速比 = 136.70 / 54.66 = **2.50×**。正文主报 N=100（四个变体在同一二进制、同一次运行内测完，可比性最强），N=1000 作为「迭代放大 10 倍结论不变」的佐证。

> 说明：`acc` 变体（`vwadd.wv` 路线）在本机 emu 上挂死（100% CPU 空转、无 GOOD TRAP，两次复现），故未纳入上表，详见 §6.5 与 `基线实现与测量.md` §6.4/§9。基线三个变体都是**手写内联汇编**，不存在编译器生成的冗余指令，所以加速比没有被人为放大。

5.4 怎么解读这些数字

（1）吞吐加速比（16×–20×）大于指令数下降（10–12 → 1），乍看反直觉，其实是香山的 FU 分组决定的：

基线用到的指令	落到的执行单元	该单元 latency（`FuConfig.scala`）
`vwmul.vv` / `vwadd.vv` / `vadd.vv`	`VialuCfg`（VIntFixpAlu）	1 拍，流水
`vslidedown.vi` ×3 / `vrgather.vv` ×1	`VppuCfg`（VPerm）	**2 拍**
`vse32.v` / `vle8.v`	无 FU，走 VLSU 拆分	`UncertainLatency`

`gat` 的 12 条里有 4 条是 `Vppu`（2 拍）且必须串行排队（跨 lane 错位天然是依赖链），而 `vdot` 的乘加全在 `Vialu` 一个通道内、1 拍出栈。**所以省下的不只是 11 条指令，还省掉了 4 次「跨通道 + 2 拍」的排队。**

（2）串行延迟只快 2.45×–2.78×，远小于吞吐的 16×–20×。原因是串行链里 `vdot` 本体只占很小一段：

```
vsetivli → vle8.v v1 ← 依赖上一轮的 pa（地址）→ vdot.vv → vsetivli → vse32.v → 标量 lw → 下一轮 pa
```

整条链 54.66 拍，其中访存（load 到 L1 命中约 4–6 拍、store→load 前递）与两条 `vsetvli` 的配置唤醒占了大头；`vdot` 只贡献 1 拍 ALU。**这正是「指令级优化在依赖链场景收益有限」的定量证据**：即使把 12 条压成 1 条，端到端延迟也只降 2.8 倍。

（3）对真实负载的意义：LLM 推理里的 QK^T、FFN 的 int8 GEMM 都是**吞吐受限**（大量独立的点积同时在场），所以应看吞吐列——16×–20×，而不是串行列。反过来说，若某段代码是单链、无并行度，那么真正该优化的是访存与配置指令，而不是再压指令条数。

（4）`vdot` 自身的 ser/thru = 54.66/6.28 = 8.7×，说明它是一条低延迟、可流水的高吞吐指令；同时它让「每 4 个 int8 乘加」的稳态成本降到 6.28 拍，其中 4.57 拍是循环骨架——**留给 rtl 优化的空间只剩 1.7 拍**，这也解释了为什么 §7 提的扩展方向是「一次算 8 个或 16 个输出」（`vdot.vv` 的 `vl` 上限此刻由 16 字节源宽度卡住），而不是继续微调加法树。

（5）局限：主口径 N=100；emu 约 190 cycles/s，N=1000 吞吐需要 60 万周期（约 1 小时），本报告 N=1000 只做了串行口径（154.69 / 136.70，与 N=100 差 +1.7%/+1.9%）；`acc` 变体因模型侧缺陷缺席，故「最坏基线」是 `ngt`；未采集 per-FU 的 uop 计数，第 (1) 点的通道解释依据的是 `FuConfig` 中的 latency 配置与指令归属，而非 perf 计数器。

6. 顺带发现的五个微架构/模型问题（探索性）

6.1 `vsext.vf2` 在 kunminghu-v2 RTL 上只按低 8 位扩展

探针程序在 emu 上逐条验证：`vsext.vf2`（e16m2 → e32m4）输入 `[0x0082, 0x00C6, 0x010C]`（130/198/268），实得 `0xFFFFFF82 / 0xFFFFFFC6 / 0xFFFFFF0C`（= `sext8(0x82)`）。中间插 8 条无关向量指令隔离流水线后结果不变，排除旁路/时序，判定为数据通路 bug。同一 RTL 上 `vwmul.vv`、`vwadd.vv`、`vadd.vv`、`vslidedown.vi`、`vrgather.vv`、`vid.v`、`vsll.vi` 全部正确。

影响：基线因此改用等价的 `vwadd.vv(+0)` 做符号扩展，指令条数不变，公平性不受影响；同时也说明本作品的 `vdot` 不能复用 `VEC_EXT2` 通路。

6.2 NEMU 未实现 `vwadd.wv`

NEMU 参考模型里 `vwadd.wv` 触发非法指令（`vwmacc.vv`/`vadd.vv` 等正常）。因此 `acc` 变体只能在 RTL 上测，NEMU 侧用 `gat`/`ngt` 对拍。这是基线的**有利偏差**（少一个竞争者），此处如实写明。

6.3 香山的 uop 拆分方式决定了 `vl ≤ 16` 的硬约束

香山对 `VEC_VVV` 按 **SEW 粒度每拍 16 字节**切 uop（VLEN=128）。`vdot` 是「16 字节源 → 4 字节目的」，若 `vl=32`，第二个 uop 的写回字节偏移会被框架算成 16，而正确值应是 4..7 → **静默写错**。因此本作品在 RTL 与

NEMU 都把 `vl>16` 判为 illegal。

这不是”实现不出来”，而是”要用 1 条指令跨 2 个 uop 就得改全核共用的拆分器”。正确的扩展方向是给 `vuopIdx` 加一层「4 源 uop → 1 目的 uop」映射（见 §6.4），而不是改拆分器本身。

6.4 若把 vdot 做成 2 拍 / 多 uop

量化估算（详见阶段二设计文档 §3.7）：16 个 s8×s8 乘法器 ≈1000–1600 LUT 或 16 个 DSP、12 个 16/18 位加法器 ≈250 LUT、32 位选通 ≈150 LUT、约 130 个触发器；关键路径 5–6 级 LUT，比香山已有 64 位加法器链路短，**不构成新的关键路径**。切 2 拍不提高吞吐（仍是每拍 1 条的发射率），只多一次写回端口竞争，故本版本用 1 拍。

6.5 `vwadd.wv` 在本机 emu 上挂死（现象确定，根因未定位）

`acc` 变体（`vwmul` 后转 e32m4，再用 `vslidedown.vi`×3 + `vwadd.wv`×3 逐列累加）**在 emu 上跑不完**：两次独立运行（N=4/8 与 N=100 两个二进制）都精确停在 `check_variant(dot_acc)` 处，最后一行是 `[B] N=...`，之后无任何输出、不是 trap、不是 GOOD TRAP，进程 100% CPU 空转，只能 `kill -9`。NEMU 侧 `vwadd.wv` 本身就是非法指令（§6.2），所以 NEMU 根本跑不到这个点。

只读排查（未改任何文件）：`VecDecoder.scala:429` 把 `VWADD_WV` 解到 `UopSplitType.VEC_WVW`，`yunsuan/package.scala:122` 有 `def vwadd_wv = LiteralCat(FMT.WVW, SINT, VialuOpcode.vadd)`，`VIAluFix.scala:61/:93` 有 WVW 的位宽拼法——译码与 op 映射都在，说明**不是”没实现”**，但卡死点无法用现有证据定位（两个微探针程序也在打印首行后挂住，未能隔离到单条指令），可能与 WVW 触发的 replay 或发射握手有关，**仅作猜测、不下结论。**

处置：`acc` 只作为源码里的备选写法保留，不进入性能表；`sw/cie-vdot-perf` 默认 `USE_VWADD_WV=0`。这条记录的价值在于：它是一处**可复现的 RTL 挂死**（而非性能差异），对香山社区是一个可直接跟进的 bug 报告线索。

7. 扩展潜力

1. **e16/e32 版本**：把 4 个 s8 换成 2 个 s16 或 1 个 s32，编码上换 `FMT`/`SINT` 与新的 `VialuOpcode` 常量，乘法树可参数化复用；
2. `vdot.vx`：与标量寄存器点积，省掉一个 `vle8.v`，对 GEMV（LLM decode 阶段就是 GEMV）更自然；
3. **4-bit 量化**：边缘 LLM 常用 int4 权重，一个 128 位 vreg 可放 32 个 int4，理论吞吐再翻倍；
4. **attention 的 QK^T**：`Q·K` 就是点积，配合 `vdot` 可把打分从 12 条降到 1 条；
5. **与 `vfwmacc` 对比**：浮点点积 1 条 `vfwmacc` 每次只累加一个元素对，整数 4 路点积在 int8 量化推理上有 4× 面积效率优势。

8. 复现步骤

```
# 0) 环境
git clone https://github.com/OpenXiangShan/xs-env && cd xs-env && sudo -s ./setup-tools.sh && source
        setup.sh

# 1) 打补丁（本仓库 rtl/ 与 nemu/ 下）
cd XiangShan && git checkout kunminghu-v2 && git apply ../rtl/xiangshan-vdot.patch
cd yunsuan   && git apply ../rtl/yunsuan-vdot.patch
cd NEMU      && git apply ../nemu/nemu-vdot.patch

# 2) 构建
cd XiangShan && make emu -j16                        # -> build/emu
cd NEMU && make riscv64-xs_defconfig && make -j4     # -> 解释器
cd NEMU && make riscv64-xs-ref_defconfig && make -j4 # -> difftest 用的 .so

# 3) 跑测试（sw/ 下有 7 个程序）
./build/emu -i <app>.bin --no-diff -C 90000
./build/emu -i <app>.bin -C 90000 --diff=../NEMU/build/riscv64-nemu-interpreter-so
```

9. 自评（对照评审细则）

维度	分值	本阶段支撑
完整性（阶段三）	25	NEMU 实现 + 协同仿真 + 基线 + 加速比，四项齐全
可行性（功能验证）	10	三层验证：C 黄金模型 / NEMU / RTL difftest
探索性（软硬件协同）	5	同一份 NEMU 代码既当 ISA 模拟器又当 RTL 参考模型；并定位两个模型缺陷
探索性（微架构优化）	5	uop 拆分约束的根因分析、2 拍方案的量化取舍
创新性（扩展潜力）	5	e16/e32/int4/vx 演进路径与 LLM 推理映射